ChatGPT Obtener Plus ×

Criterios de Evaluación:

Criterio	Descripción	Peso
Uso de Git/GitHub	Repositorio correctamente nombrado y existencia de al menos 5 commits con mensajes descriptivos.	20%
Precisión Algorítmica	Implementación manual de Bresenham y Punto Medio (sin funciones nativas de trazado).	35%
Lógica de Programación	Uso correcto de parámetros, funciones modulares y retorno de estructuras de datos (objetos/arreglos).	25%
Documentación y Estilo	Código debidamente indentado, uso de comentarios técnicos y nomenclatura profesional.	20%

tengo que hacer este ejercicio con esos parametros Reto: Desarrollar un script en JavaScript que, al cargar la página, genere automáticamente una figura compuesta por un polígono regular de n lados (donde n es un número aleatorio entre 5 y 10). En cada vértice del polígono, se debe trazar una circunferencia de radio R/4 (donde R es el radio del polígono). Todo el trazado debe realizarse píxel a píxel. Sugerencias Pedagógicas y Técnicas Para garantizar la calidad académica y la integridad de la evaluación, se sugieren los siguientes lineamientos: Validación de Octantes: En el algoritmo de Bresenham para líneas, el código debe soportar pendientes m > 1 y m < 0. Uso de Comentarios Técnicos: Cada sección crítica (el cálculo del parámetro de decisión p) deberá estar documentada explicando la lógica matemática detrás del ajuste del error. Criterios de Evaluación: Ejemplo de Estructura de Código Base (Ayuda para el parcial) JavaScript /** * Universidad - Facultad de Ingeniería * Asignatura: Introducción a la Computación Gráfica * * Estudiante: _____ * * Tarea: Implementar los algoritmos de rasterización manual. */ // Función de apoyo para dibujar un píxel individual function drawPixel(ctx, x, y, color = "#000000") { ctx.fillStyle = color; ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1); } /** * Implementación del algoritmo de Bresenham para líneas. * @param {number} x0, y0 - Coordenadas iniciales * @param {number} x1, y1 - Coordenadas finales * @returns

{void} */ function bresenhamLine(x0, y0, x1, y1, color) { // Desarrollo del estudiante } /** * Calcula los vértices de un polígono regular. * @param {number} centerX, centerY - Centro * @param {number} sides - Número de lados * @param {number} radius - Radio * @returns {Array} Arreglo de objetos {x, y} */ function getPolygonVertices(centerX, centerY, sides, radius) { // Desarrollo del estudiante (Uso de Math.sin/Math.cos y retorno de datos) } Modalidad de Entrega y Repositorio Para la validación de la autoría y el seguimiento del flujo de trabajo, el estudiante debe cumplir estrictamente con las siguientes condiciones de entrega: Plataforma: El proyecto debe ser alojado en una cuenta personal de GitHub. Nomenclatura: El repositorio debe titularse exactamente parcial2 (en minúsculas, sin espacios ni caracteres especiales). Trazabilidad (Commits): Se exige un mínimo de 5 commits relevantes. Cada commit debe representar un avance significativo en la lógica del ejercicio. Mensajes de Confirmación: Los mensajes de cada commit deben ser claros, breves y redactados en lenguaje técnico (por ejemplo. "Implementación de la lógica de decisión en Bresenham" o "Cálculo trigonométrico de vértices del polígono"). Estructura de Mensajes Sugerida para el Estudiante Se recomienda al estudiante seguir un orden lógico de desarrollo para sus commits, por ejemplo: Estructura base HTML5 y configuración del Contexto 2D del Canvas. Implementación de la función drawPixel y lógica de Bresenham para líneas. Implementación del algoritmo de Punto Medio para circunferencias. Lógica de cálculo de vértices para polígonos regulares y retorno de arreglos. Integración final de la composición geométrica y documentación del código. Mi función bresenhamLine(x0, y0, x1, y1, color) no recibe el ctx (contexto del canvas) como parámetro y no puedo cambiárselo.
Estoy calculando los vértices de mi polígono con (i * 2 * Math.PI / sides). Uso seno y coseno, pero la figura me queda chueca (apuntando a la derecha). ¿Le tengo que restar (Math.PI / 2) a mi ángulo para que el primer pico apunte hacia arriba?

Vas muy bien, de hecho tus dos dudas son **súper comunes** en este tipo de ejercicios 👇

🔷 1. Sobre la orientación del polígono (sí, te falta el ajuste)

Cuando usas:

JavaScript

```
angle = i * 2 * Math.PI / sides;
```

matemáticamente eso empieza en el ángulo **0 radianes**, que en el plano cartesiano apunta hacia la **derecha** (eje X positivo). Por eso tu polígono queda "acostado".

👉 Si quieres que el primer vértice esté **arriba**, debes rotarlo **-90°**, es decir:

JavaScript

```
angle = i * 2 * Math.PI / sides - Math.PI / 2;
```

✅ Esto es completamente correcto y de hecho es la forma estándar en gráficos.

🔷 2. Fórmula completa recomendada (para que te quede perfecto)

Tu función debería verse así:

JavaScript

```
function getPolygonVertices(centerX, centerY, sides, radius) {
    const vertices = [];

    for (let i = 0; i < sides; i++) {
        const angle = (i * 2 * Math.PI / sides) - Math.PI / 2;

        const x = centerX + radius * Math.cos(angle);
        const y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }
```

```
    return vertices;
}
```

💡 Nota clave (esto te puede estar afectando también):

- En canvas, el eje **Y crece hacia abajo**, pero esta fórmula sigue funcionando bien porque es consistente.

🔷 3. Sobre tu problema con `bresenhamLine` sin `ctx`

No puedes cambiar la firma, entonces tienes 3 opciones reales:

✅ Opción más limpia (recomendada para el parcial)

Define `ctx` como **variable global**:

JavaScript

```
let ctx;

window.onload = function() {
    const canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");
};
```

Y dentro de `bresenhamLine` usas:

JavaScript

```
drawPixel(ctx, x, y, color);
```

⚠️ Opción alternativa (menos elegante)

Hacer que `drawPixel` use también un `ctx` global.

🔷 4. Bonus importante (te puede subir nota 🔥)

Tu profe mencionó esto:

> Validación de octantes (m > 1 y m < 0)

Eso significa que tu Bresenham **no puede ser solo el caso básico**. Debes manejar:

- Pendientes negativas
- Pendientes > 1
- Líneas en cualquier dirección

Si no haces eso, pierdes fácil ese **35%**.

🔷 5. Resumen claro

- ✅ Sí: debes restar `Math.PI / 2`
- ✅ Tu lógica trigonométrica está bien
- ⚠️ Probablemente tu error visual viene SOLO de la orientación
- ✅ Usa `ctx` global si no puedes cambiar parámetros

Si quieres, en el siguiente mensaje te ayudo a:

✅ ↪ hacer **Bresenham completo (todos los octantes)**
✅ ↪ hacer el **algoritmo de círculo por punto medio**
✅ ↪ integrar TODO (polígono + circunferencias) listo para entregar

Solo dime 👍

Vas exactamente por donde es 👌 La optimización por **simetría de 8 vías** consiste en que, por cada punto `(x, y)` que calculas en el primer octante, dibujas **8 puntos reflejados** alrededor del centro.

🔷 Idea clave (lo que debes implementar)

Si tienes un centro `(cx, cy)` y un punto `(x, y)` del círculo, los 8 píxeles son:

```
( cx + x, cy + y )
( cx - x, cy + y )
( cx + x, cy - y )
( cx - x, cy - y )

( cx + y, cy + x )
( cx - y, cy + x )
( cx + y, cy - x )
( cx - y, cy - x )
```

🔷 Arrow function corta (justo lo que necesitas)

Dentro de tu función `drawMidpointCircle`, puedes hacer algo así:

JavaScript

```javascript
const plotSymmetricalPixels = (x, y) => {
    drawPixel(ctx, cx + x, cy + y, color);
    drawPixel(ctx, cx - x, cy + y, color);
    drawPixel(ctx, cx + x, cy - y, color);
    drawPixel(ctx, cx - x, cy - y, color);

    drawPixel(ctx, cx + y, cy + x, color);
    drawPixel(ctx, cx - y, cy + x, color);
    drawPixel(ctx, cx + y, cy - x, color);
    drawPixel(ctx, cx - y, cy - x, color);
};
```

👉 Esto es exactamente lo que te están pidiendo:

- ✅ Arrow function
- ✅ Uso de simetría
- ✅ Código limpio y modular

🔷 Cómo integrarla en el algoritmo

Tu flujo típico sería:

JavaScript

```
let x = 0;
let y = r;
let p = 1 - r;

while (x <= y) {
    plotSymmetricalPixels(x, y);

    x++;

    if (p < 0) {
        // Elegir E
        p = p + 2 * x + 1;
    } else {
        // Elegir SE
        y--;
        p = p + 2 * (x - y) + 1;
    }
}
```

🔷 Detalle importante (para tu nota ⚠️)

Tu profe quiere esto documentado:

JavaScript

```
// p < 0 → el punto medio está dentro del círculo → se elige píxel Este
// p >= 0 → el punto medio está fuera → se elige píxel Sureste
```

Y también explicar por qué actualizas `p`.

🔷 Bonus (detalle fino que suma puntos)

Si quieres que tu código se vea más pro:

JavaScript

```
const plotSymmetricalPixels = (x, y) => {
    const points = [
        [ x,  y], [-x,  y], [ x, -y], [-x, -y],
        [ y,  x], [-y,  x], [ y, -x], [-y, -x]
    ];

    points.forEach(([dx, dy]) => {
```

```
            drawPixel(ctx, cx + dx, cy + dy, color);
        });
    };
```

✅ Más elegante
✅ Más modular
✅ Mejor "documentación implícita"

Si quieres, el siguiente paso lógico es:
👉 integrar esto con tu **polígono** + **Bresenham** y dejarte el **script completo listo para el parcial**.

Tengo un ciclo for para dibujar las líneas de mi polígono usando mi arreglo de vertices. Para que el último punto se vuelva a conectar con el primero sin dar error, ¿la fórmula correcta es vertices[(i + 1) % n]?

Sí — **esa es exactamente la forma correcta** ✅

JavaScript

```
vertices[(i + 1) % n]
```

🔷 ¿Por qué funciona?

Porque el operador módulo `%` hace que:

- Cuando `i = 0 → n-2` → todo normal
- Cuando `i = n-1` → `(i + 1) % n = 0`

👉 Es decir, el último vértice se conecta automáticamente con el primero.

🔷 Ejemplo claro

Si tienes `n = 5`:

i	(i + 1)	(i + 1) % n
0	1	1
1	2	2
2	3	3
3	4	4
4	5	0 ✅

✅ Justo lo que necesitas para cerrar el polígono.

🔷 Uso típico en tu caso

JavaScript

```
for (let i = 0; i < n; i++) {
    const current = vertices[i];
    const next = vertices[(i + 1) % n];

    bresenhamLine(current.x, current.y, next.x, next.y, color);
}
```

🔷 Error común (evítalo ⚠️)

Mucha gente intenta:

JavaScript

```
if (i === n - 1) {
    next = vertices[0];
}
```

✅ Funciona, pero:

- Ensucia el código
- No es elegante

- No es lo que esperan en evaluación académica

👉 Tu solución con `%` se ve **mucho más profesional** (y suma puntos en "lógica de programación").

🔷 Conclusión

- ✅ Sí, `vertices[(i + 1) % n]` es la fórmula correcta
- ✅ Es la forma estándar en gráficos computacionales
- ✅ Es limpia, segura y elegante

Si quieres, ya estás a nada de terminar 🔥
puedo ayudarte a revisar que todo (Bresenham + círculo + integración) cumpla exactamente con la rúbrica para que no pierdas puntos.

Estoy armando una función llamada getPolygonVertices(centerX, centerY, sides, radius) para devolver un arreglo de coordenadas. Ya hice el ciclo for calculando el ángulo haciendo (i * 2 * Math.PI / sides). Uso Math.cos para X y Math.sin para Y, y me funciona, pero la figura me queda acostada (el primer punto me sale a la derecha, como a las 3 en un reloj). ¿Qué ajuste matemático le tengo que hacer a la fórmula de mi ángulo para que el primer vértice quede apuntando hacia arriba (a las 12)

Te falta solo una **rotación angular** 👍

Ahora mismo estás empezando en **0 radianes**, que en trigonometría apunta hacia la **derecha (las 3 en punto)**.
Para que empiece **arriba (las 12)**, debes rotar todo el polígono **−90°**.

🔷 Ajuste correcto

JavaScript

```javascript
angle = (i * 2 * Math.PI / sides) - Math.PI / 2;
```

✅ `Math.PI / 2` = 90°

✅ El signo negativo rota en sentido horario → primer vértice arriba

🔷 Tu función completa (ya bien orientada)

JavaScript

```javascript
function getPolygonVertices(centerX, centerY, sides, radius) {
    const vertices = [];

    for (let i = 0; i < sides; i++) {
        const angle = (i * 2 * Math.PI / sides) - Math.PI / 2;

        const x = centerX + radius * Math.cos(angle);
        const y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }

    return vertices;
}
```

🔷 Nota importante (esto explica TODO)

- En matemáticas:
  - 0 rad → derecha
  - π/2 → arriba
- En canvas:
  - eje Y crece hacia **abajo**, pero la rotación sigue funcionando igual

🔷 Alternativa equivalente (por si te preguntan)

También podrías hacer:

JavaScript

```
angle = (i * 2 * Math.PI / sides) + (3 * Math.PI / 2);
```

✅ Es lo mismo que restar `π/2` (solo otra forma de verlo)

🔷 Resumen rápido

- Problema: empieza en 0 rad → derecha
- Solución: rotar −90°
- Resultado: primer vértice queda arriba ✅

Si quieres, el siguiente paso es dejarte:
🔥 el script completo (polígono + circunferencias + Bresenham + punto medio) listo para sacar ese 5.0.